Module – 3

Consensus Mechanisms & Security Models: Proof of Work (PoW), Proof of Stake (PoS) & its variants (DPoS, NPoS), Byzantine Fault Tolerance (PBFT, Tendermint), Blockchain fork attacks and mitigation strategies.

Proof of Work (PoW) & its variants

Proof of Work (PoW) is a foundational consensus mechanism in blockchain technology. It requires substantial computational effort to validate transactions, ensuring the security and integrity of the blockchain network without reliance on a trusted third party. Initially popularized by Bitcoin, PoW underpins many cryptocurrencies by facilitating secure peer-to-peer transaction processing.

The term "proof of work" was first used by **Markus Jakobsson** and **Ari Juels** in a publication in 1999.

Cryptocurrencies like Litecoin, and Bitcoin are currently using PoW. Ethereum was using PoW mechanism, but now shifted to Proof of Stake (PoS).

Purpose of PoW

The **purpose** of a consensus mechanism is to bring all the nodes in agreement, that is, trust one another, in an environment where the nodes don't trust each other.

- All the transactions in the new block are then validated and the new block is then added to the blockchain.
- The block will get added to the chain which has the longest block height (see blockchain forks to understand how multiple chains can exist at a point in time).
- Miners (special computers on the network) perform computation work in solving a complex mathematical problem to add the block to the network, hence named, Proof-of-Work.
- With time, the mathematical problem becomes more complex.

Features of PoW

There are mainly two features that have contributed to the wide popularity of this consensus protocol, and they are:

- It is hard to find a solution to a mathematical problem.
- It is easy to verify the correctness of that solution.

How Does PoW Work?

The PoW consensus algorithm involves verifying a transaction through the mining process.

Mining:

The Proof of Work consensus algorithm involves solving a computationally challenging puzzle in order to create new blocks in the Bitcoin blockchain. The process is known as 'mining', and the nodes in the network that engages in mining are known as 'miners'.

- The incentive for mining transactions lies in economic payoffs, where competing miners are rewarded with 6.25 bitcoins and a small transaction fee.
- This reward will get reduced by half its current value with time.

How PoW Works (Step-by-Step)

Step 1: Transaction Broadcast

Users initiate transactions (e.g., sending cryptocurrency), which are broadcast to the network.

Step 2: Block Formation

Miners collect transactions into a candidate block.

Step 3: Puzzle Solving (Hashing)

Miners try to find a value called a **nonce** such that the block's hash meets a specific condition.

This is based on a cryptographic hash function like:

- **SHA-256**

Miners repeatedly compute hashes:

Hash(Block Data + Nonce) → Output

They must find a hash with a certain number of leading zeros.

Step 4: Proof Submission

The first miner to find a valid hash broadcasts the solution.

Step 5: Verification

Other nodes quickly verify the solution (easy to check, hard to find).

Step 6: Block Addition

If valid, the block is added to the blockchain, and the miner gets rewarded.

DPoS (Delegated Proof of Stake)

Delegated Proof of Stake (DPoS) is a consensus mechanism designed to improve the efficiency and scalability of traditional Proof of Stake systems. Instead of every participant validating transactions, users vote for a small group of trusted delegates who maintain the blockchain on their behalf.

1. Core Idea of DPoS

DPoS introduces a democratic layer:

- Token holders → vote
- Elected delegates (also called witnesses/block producers) → validate transactions and create blocks

It's often described as a "representative democracy" in blockchain.

How DPoS Works (Step-by-Step)

Step 1: Token Ownership

Users hold cryptocurrency tokens in the network.

Step 2: Voting for Delegates

- Each token holder can vote for delegates
- Voting power is proportional to the number of tokens held (More tokens = more voting influence)

Step 3: Delegate Selection

- Top N delegates (e.g., 21 or 101 depending on the blockchain) are selected
- These delegates are responsible for validating transactions

Example platforms:

- EOS
- TRON
- Steem

Step 4: Block Production

- Delegates take turns producing blocks in a fixed order
- Each delegate gets a time slot
- If a delegate fails, the next one steps in

Step 5: Rewards Distribution

- Delegates earn rewards for block creation
- They may share rewards with voters (incentive mechanism)

Step 6: Continuous Voting

- Voting is ongoing
- Poor-performing or malicious delegates can be voted out and replaced instantly

Advantages of DPoS

This popular evolution of the PoS concept has several advantages:

- Accessibility: Because there is no need for specialized and expensive equipment, anyone can become a delegate.
- Democracy: The low barrier to entry allows more people to engage in a consensus process, making it democratic and financially inclusive.
- Scalability: Due to the network having a limited number of delegates, it allows for faster consensus, which means improved performance.
- Environmentally Friendly: It doesn't require much power to run the network, making it more sustainable.

Disadvantages of DPoS

Despite its significant advantages, Delegated Proof-of-Stake also has its fair share of limitations.

NPoS (Nominated Proof of Stake)

NPoS combines economic security with social trust by allowing users to nominate validators. The process involves nomination, validator selection, block production, and incentives for honest behavior. Key benefits include enhanced security, greater decentralization, and energy efficiency compared to Proof of Work.

Nominated Proof of Stake (NPoS) is an advanced blockchain consensus mechanism that enhances the traditional Proof of Stake (PoS) model by introducing the concept of nomination, enabling greater decentralization, security, and community participation. This article explores how NPoS works, its benefits, its growing adoption within the industry, and what it means for blockchain users and security enthusiasts.

How Does NPoS Work?

The NPoS process unfolds in several steps:

1. **Nomination:** Token holders choose one or more validators to support by staking their assets with them. This is a vote of confidence in the validator's reliability and conduct.
2. **Validator Selection:** Validators are selected using a **verifiable random function**, ensuring fairness and transparency in validator elections. The likelihood of selection increases with the size of the stake backing a given validator.
3. **Block Production:** Once chosen, validators take turns in a randomized sequence to produce blocks. This prevents any single validator from dominating block creation and maintains network fairness.
4. **Incentives and Punishments:** Honest validators and their nominators are rewarded for correct behavior, while malicious actors are penalized through a process known as **slashing**—losing a portion of their staked assets.

Key Benefits of NPoS

- **Enhanced Security:** By requiring both validators and nominators to stake assets and by penalizing malicious behavior, NPoS raises the cost of attacks and increases protection against various threats.
- **Greater Decentralization:** The nomination system distributes influence among a wider community, reducing the risk of centralization present in some PoS and PoW systems.
- **Efficient and Sustainable:** NPoS eliminates energy-intensive mining, making it a more environmentally sustainable choice compared to PoW protocols.

What is Byzantine Fault Tolerance?

Byzantine Fault Tolerance(BFT) is the feature of a distributed network to reach consensus (agreement on the same value) even when some of the nodes in the network fail to respond or respond with incorrect information. The objective of a BFT mechanism is to safeguard against the system failures by employing collective decision making(both - correct and faulty nodes) which aims to reduce to influence of the faulty nodes. BFT is derived from Byzantine Generals' Problem.

Origin of the Concept

BFT comes from the famous Byzantine Generals Problem, where multiple generals must coordinate an attack, but some may be traitors sending false messages. The challenge is to ensure all loyal generals agree on the same plan.

What Problem Does BFT Solve?

In distributed systems, nodes may:

- Crash or stop responding
- Send incorrect or conflicting data
- Act maliciously (hacking, tampering)

BFT ensures the system still:

- Makes correct decisions
- Maintains consistency
- Avoids failure or confusion

Types of Faults

1. **Crash faults** – node stops working
2. **Omission faults** – messages not sent/received
3. **Byzantine faults** – arbitrary/malicious behaviour

BFT specifically handles the most difficult case: **Byzantine faults**.

How BFT Works (Conceptually)

- Nodes exchange messages with each other
- They verify consistency of information
- Decision is made based on **majority agreement**
- Faulty nodes are ignored if they conflict with the majority

Why BFT is Important

- Ensures trust in untrusted environments
- Critical for blockchain systems
- Maintains data integrity and consistency
- Enables secure distributed computing

Real-World Applications

- Blockchain systems
- Distributed databases
- Aerospace and military systems
- Financial networks

Examples of systems using BFT or its variants:

- Bitcoin (uses probabilistic BFT via Proof of Work)
- Ethereum (moving toward PoS-based BFT models)
- Hyperledger Fabric (uses pBFT-like mechanisms)

practical Byzantine Fault Tolerance(pBFT)

It is difficult to form a consensus in a distributed system even when some nodes fail or respond with faulty values (malicious nodes). It is known as the Byzantine generals' problem. Since this problem's inception, many consensus algorithms have been made which are Byzantine fault-tolerant (when correctly working nodes in a system can reach an agreement). Practical Byzantine fault tolerance(pBFT) is also one of the consensus algorithms introduced to solve the Byzantine generals' problem. It is designed to allow a distributed system (like a blockchain or distributed database) to reach agreement even when some nodes behave maliciously or unpredictably (i.e., exhibit Byzantine faults).

How does pBFT work?

The pBFT algorithm provides practical state machine replication in which all nodes are sequentially ordered, with one node as the primary node and others as secondary nodes (backup/replica nodes). A pBFT can function on the condition that the total number of malicious nodes must be less than one-third of the total nodes ***n*** in the system.

The pBFT consensus happens in the following five steps:

- The client makes a request and sends it to the primary node.
- The primary node broadcasts the request to all the secondary(backup) nodes. It is called the pre-prepare phase.
- Then every node (primary and secondary) sends a prepare message to all other nodes.
- Once every node receives (n/3)+1 prepare messages, it sends a commit message to all other nodes and commits the changes made by the client's request.
- Once every node receives (n/3)+1 commit messages from other nodes, it sends a reply to the client.

Advantages of pBFT

The following are a few advantages of using pBFT over other consensus algorithms:

1. **Energy efficient:** pBFT does not need to compute complex mathematical problems to reach a consensus. Therefore, it is much faster and energy efficient than algorithms such as proof of work.

2. **Faster transaction finality:** Consensus algorithms such as proof of work require confirmations from other nodes before adding it to their journal (which takes 10–60 minutes). However, pBFT does not require any confirmations, it is much more efficient than other consensus algorithms.

3. **Low reward variance:** In pBFT, every node is participating in processing the client's request. Hence, every node can be incentivized, resulting in low reward variance and a more fair reward system.

Limitations of pBFT

Some limitations of pBFT are as follows:

1. **Sybil attacks:** A distributed network using pBFT is susceptible to Sybil attacks if one entity controls many nodes in the network. As the number of nodes increases, it becomes difficult to carry out a Sybil attack.

2. **Scalability:** As the number of nodes in the network increases, communication overhead (messages sent to other nodes) increases. It is shown by the equation, $O(n^k)$, where n is the messages and kk is the number of nodes.

Blockchain fork attacks

Basically, forks are divided into two categories i.e. Codebase Fork and Live Blockchain Fork. And then Live Blockchain Fork is divided into further two parts i.e. Intentional Fork and Accidental Fork, as you can see in the above-mentioned figure the Intentional fork is then further divided into two parts i.e. Soft Fork and Hard Fork.

TYPES OF FORKS:

CODEBASE FORK: In codebase blockchain fork you can copy the entire code of a particular software. Let us take BITCOIN as an example, so suppose you copied the whole blockchain code and modified it according to your need, say that you decreased the block creation time, made some crucial changes and created a faster software than BITCOIN and publish / launch it has a new whole software named against you, by completing the whole white paper work process. So, in these way a new BLOCKCHAIN will be created from an empty blank ledger. It's a fact that many of these ALT COINS which are now running on the blockchain are been made in this way only by using the codebase fork i.e. they have made little up and down changes in the code of BITCOIN and created their whole new ALT COIN.

LIVE BLOCKCHAIN FORK: Live Blockchain fork means a running blockchain is been divided further into two parts or two ways. So, in live blockchain at a specific page the software is same and from that specific point the chain is divided into two parts. So, in context to this fork the Live Blockchain Fork can occur because of two reasons:

- **ACCIDENTAL FORK / TEMPORARY FORK:** When multiple miners mine a new block at nearly the same time, the entire network may not agree on the choice of the new block. Some can accept the block mined by one party, leading to a different chain of blocks from that point onward while others can agree on the other alternatives (of blocks) available. Such a situation arises because it takes some finite time for the information to propagate in the entire blockchain network and hence conflicted opinions can exist regarding the chronological order of events. In this fork, two or more blocks have the same block height. Temporary forks resolve themselves eventually when one of the chains dies out (gets orphaned) because majority of the full nodes choose the other chain to add new blocks to and sync with. Example (TEMPORARY FORK / ACCIDENTAL FORK): Temporary forks happen more often than not and a usual event that triggers this fork is mining of a block by more than one party at nearly the same time.
- **INTENTIONAL FORK:** In intentional fork the rules of the blockchain are been changed, knowing the code of the software and by modifying it intentionally. This gives rise to two types of forks which can occur based on the backwards-compatibility of the blockchain protocol and the time instant at which a new block is mined. So Intentional fork can be of two types:
  - **SOFT FORK:** When the blockchain protocol is altered in a backwards-compatible way. In soft fork you tend to add new rules such that they do not clash with the old rules. That means there is no connection between the old rules and new rules. Rules in soft fork are tightened. When there is a change in the software that runs on the nodes (better called as

'full nodes') to function as a network participant, the change is such that the new blocks mined on the basis of new rules (in the Blockchain protocol) are also considered valid by the old version of the software. This feature is also called as backwards-compatibility. Example (SOFT FORK): The Bitcoin network's SegWit update added a new class of addresses (Bech32). However, this didn't invalidate the existing P2SH addresses. A full node with a P2SH type address could do a valid transaction with a node of Bech32 type address.

- **HARD FORK:** When the blockchain protocol is altered in a non-backwards-compatible way. Hard fork is opposite of Soft fork, here the rules are loosened. When there is a change in the software that runs on the full nodes to function as a network participant, the change is such that the new blocks mined on the basis of new rules (in the Blockchain protocol) are not considered valid by the old version of the software. When hard forks occur, new currency come into existence (with valid original currency) like in the case of Ethereum (original : Ethereum, new : Ethereum Classic) and Bitcoin (original : Bitcoin, new: Bitcoin cash). Equivalent quantity of currency is distributed to the full nodes who choose to upgrade their software so that no material loss occurs. Such hard forks are often contentious (generating conflicts in the community). The final decision to join a particular chain rest with the full node. If chosen to join the new chain, the software has to be upgraded to make newer transactions valid while the nodes who do not choose to upgrade their software continue working the same. Example (HARD FORK): The new Casper update in the Ethereum Blockchain in which the consensus protocol will change from a type of Proof of Work (PoW) to a type of Proof of Stake (PoS). The nodes which install the Casper update will use the new consensus protocol. Full nodes that do not choose to install the Casper update will become incompatible with the full nodes that do.

Reasons for the occurrence of a blockchain fork:

- **Add new functionality:** The Blockchain code is upgraded regularly. Since most public blockchains are open source, it is developed by people from around the world. The improvements, issues are created, resolved and new versions are released when the time is suitable.
- **Fix security issues:** Blockchain (and cryptocurrency on top of it) is a relatively new technology as compared to the traditional currency (notes, coins, cheque), research is still underway to fully understand it. So, versions are bumped and updates are released to fix the security issues that arise in the way.
- **Reverse transactions:** The community can actually void all the transaction of a specific period if they are found to be breached and malicious.

Blockchain mitigation strategies.

Security Implications of Blockchain Forks

Blockchain forks, particularly hard and contentious forks, introduce several security risks. Understanding these risks is essential for designing robust mitigation strategies.

1. Double-Spending Attacks

During a fork, competing chains create opportunities for malicious actors to spend the same assets on both chains. This is especially common in the immediate aftermath of a fork before consensus stabilizes.

Case Study:

The 2018 Bitcoin Gold attack exploited chain vulnerabilities, resulting in double-spending losses exceeding $18 million.

Mitigation:

Use of replay protection to ensure transactions are valid only on one chain.

Strengthening consensus mechanisms like Proof of Stake (PoS) or hybrid PoW/PoS models.

2. 51% Attacks

A 51% attack occurs when a malicious entity controls the majority of a network's hash rate, enabling them to manipulate transaction history. Forks, especially contentious ones, can weaken the hash rate on each chain, making smaller chains vulnerable.

Case Study:

Ethereum Classic experienced multiple 51% attacks after its split from Ethereum, highlighting vulnerabilities in reduced-hash-rate networks.

Mitigation:

Diversify mining pools to prevent concentration of hash power.

Introduce penalties for chain reorganization attempts.

3. Replay Attacks

When two chains share the same transaction signatures, a transaction valid on one chain can be maliciously replayed on the other.

Mitigation:

Implement strong replay protection mechanisms by assigning unique signatures to each chain.

4. Lost Funds and Wallet Incompatibility

Post-fork, users often struggle with wallets or exchanges that fail to support the new chain. This can result in inaccessible funds or lost assets.

Mitigation:

Educate users and provide clear instructions for managing post-fork assets.

Ensure wallets and exchanges adopt robust support systems for forked chains.

5. Network Instability

Forks disrupt the blockchain network, often resulting in delays, reduced transaction throughput, and increased vulnerability to attacks.

Mitigation:

Pre-fork testing and simulation to assess the network's resilience.

Gradual deployment of changes to ensure stability.